

# eda-final-code

March 3, 2025

```
[1]: import pandas as pd
df = pd.read_csv("/kaggle/input/equity-post-HCT-survival-predictions/train.csv")

[2]: df = df.drop(columns=['efs_time'])

[3]: # Separate features (X) and target (y)
X = df.drop(columns=['efs']) # Drop the target column from features
y = df['efs'] # Store the target column separately

[4]: # Assume X_transformed and y are your DataFrames/Arrays

# Define the split ratio (e.g., 80% for training, 20% for testing)
train_size = int(0.8 * len(X))

# Split the data manually based on the index
X_train = X[:train_size]
X_test = X[train_size:]

y_train = y[:train_size]
y_test = y[train_size:]

[5]: X_train.shape, y_train.shape

[5]: ((23040, 58), (23040,))

[6]: from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.impute import KNNImputer
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, Dropout
from xgboost import XGBRegressor, XGBClassifier
```

```

# Define the XGBoostImputer (as previously provided)
class XGBoostImputer(BaseEstimator, TransformerMixin):
    def __init__(self, model_type='regressor', **kwargs):
        # Choose model type for regression or classification
        if model_type == 'regressor':
            self.model = XGBRegressor(**kwargs)
        elif model_type == 'classifier':
            self.model = XGBClassifier(**kwargs)
        else:
            raise ValueError("model_type should be 'regressor' or 'classifier'")
        self.model_type = model_type

    def fit(self, X, y=None):
        self.X_train = X
        self.mask = np.isnan(X) # Identify missing values
        self.X_train_no_nan = X[~self.mask.any(axis=1)] # Remove rows with NaN
        ↪values
        return self

    def transform(self, X):
        # Create a copy of X to fill the missing values
        X_filled = X.copy()
        missing = np.isnan(X_filled) # Mask of missing values

        # Iterate over each column and impute missing values
        for col in range(X_filled.shape[1]):
            if missing[:, col].any(): # If any values are missing in the column
                # Identify the missing indices for this column
                missing_indices = missing[:, col]
                X_impute = X_filled[~missing_indices] # Non-missing rows for
                ↪training the model
                y_impute = X_impute[:, col] # Use the column values as the
                ↪target (for imputation)

                # Train the model on the non-missing data
                self.model.fit(X_impute, y_impute)

                # Impute the missing values by predicting them using the
                ↪trained model
                X_filled[missing_indices, col] = self.model.
                ↪predict(X_filled[missing_indices, :]) # Use the missing rows

        return X_filled

# Define the neural network model

```

```

def create_neural_network(input_dim):
    model = Sequential()

    # First hidden layer with 128 neurons and ReLU activation
    model.add(Dense(128, input_dim=input_dim, activation='relu'))
    model.add(Dropout(0.3)) # Increased dropout to 0.3 to prevent overfitting

    # Second hidden layer with 64 neurons and ReLU activation
    model.add(Dense(64, activation='relu'))
    model.add(Dropout(0.3)) # Dropout to prevent overfitting

    # Third hidden layer with 32 neurons and ReLU activation
    model.add(Dense(32, activation='relu'))
    model.add(Dropout(0.3)) # Dropout to prevent overfitting

    # Fourth hidden layer with 16 neurons and ReLU activation
    model.add(Dense(16, activation='relu'))
    model.add(Dropout(0.3)) # Dropout to prevent overfitting

    # Output layer (for regression)
    model.add(Dense(1))

    # Compile the model with a different optimizer and a learning rate scheduler
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                  loss='mean_squared_error')

    return model

# Assuming X is your dataset
# Separate numerical and categorical columns
numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns
categorical_cols = X.select_dtypes(include=['object']).columns

# Preprocessing pipeline for numerical features
numerical_pipeline = Pipeline(steps=[
    ('scaler', StandardScaler()), # Standardize numerical features
    ('knn_imputer', KNNImputer(n_neighbors=5, weights='uniform')), # KNN_
    ↪imputation for numerical features
    ('xgboost_imputer', XGBoostImputer(model_type='regressor',
    ↪random_state=42)) # XGBoost imputation for numerical features
])

# Preprocessing pipeline for categorical features
categorical_pipeline = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse=False)), #_
    ↪One-hot encoding for categorical variables

```

```

        ('xgboost_imputer', XGBoostImputer(model_type='classifier',
        ↪random_state=42)) # XGBoost imputation for categorical features
    ])

# Column transformer to apply transformations to the appropriate columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_pipeline, numerical_cols),
        ('cat', categorical_pipeline, categorical_cols)
    ]
)

# Define a custom wrapper for the neural network to fit into the sklearn
↪pipeline
class KerasRegressorWrapper(BaseEstimator):
    def __init__(self, input_dim):
        self.input_dim = input_dim
        self.model = create_neural_network(input_dim)

    def fit(self, X, y=None):
        self.model.fit(X, y, epochs=10, batch_size=32, verbose=0)
        return self

    def predict(self, X):
        return self.model.predict(X)

# Adjust the input_dim dynamically after preprocessing
# Final pipeline with imputation and deep learning model
pipeline_nn = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', KerasRegressorWrapper(input_dim=None)) # Neural network
    ↪model
])

# Fit the pipeline with proper input_dim
X_transformed = preprocessor.fit_transform(X_train) # Apply preprocessing
    ↪steps to get transformed data
pipeline_nn.named_steps['classifier'].input_dim = X_transformed.shape[1] #
    ↪Update input_dim based on transformed shape

# Fit the pipeline
pipeline_nn.fit(X_train, y_train)

```

/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/\_encoders.py:868:  
FutureWarning: `sparse` was renamed to `sparse\_output` in version 1.2 and will

be removed in 1.4. `sparse\_output` is ignored unless you leave `sparse` to its default value.

```
warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_encoders.py:868:
FutureWarning: `sparse` was renamed to `sparse_output` in version 1.2 and will
be removed in 1.4. `sparse_output` is ignored unless you leave `sparse` to its
default value.
warnings.warn(
```

```
[6]: Pipeline(steps=[('preprocessor',
                      ColumnTransformer(transformers=[('num',
                                                       Pipeline(steps=[('scaler',
                                                                           StandardScaler()),
                                                                           ('knn_imputer',
                                                                           KNNImputer()),
                                                                           ('xgboost_imputer',
                                                                           XGBoostImputer())])),
                      Index(['ID',
                              'hla_match_c_high', 'hla_high_res_8', 'hla_low_res_6',
                              'hla_high_res_6', 'hla_high_res_10', 'hla_match_dqb1_high',
                              'hla_nmdp_6', 'hla_match_c_low', 'hla_match_drb1...',
                              'prod_type', 'cyto_score_detail', 'conditioning_intensity', 'ethnicity',
                              'obesity', 'mrd_hct', 'in_vivo_tcd', 'tce_match', 'hepatic_severe',
                              'prior_tumor', 'peptic_ulcer', 'gvhd_proph', 'rheum_issue', 'sex_match',
                              'race_group', 'hepatic_mild', 'tce_div_match', 'donor_related',
                              'melphalan_dose', 'cardiac', 'pulm_moderate'],
                              dtype='object'))]),
                      ('classifier', KerasRegressorWrapper(input_dim=214))])
```

```
[7]: # Predict and evaluate
from sklearn.metrics import mean_squared_error, r2_score
y_pred = pipeline_nn.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("mean squared error :",mse)
print("r2 score :",r2)
```

```
180/180          0s 1ms/step
mean squared error : 0.2065826929389601
r2 score : 0.16831870917591318
```

```
[8]: import pandas as pd
t = pd.read_csv('/kaggle/input/equity-post-HCT-survival-predictions/test.csv')
t.shape
```

```
[8]: (3, 58)
```

```
[9]: final_pred = pipeline_nn.predict(t)
```

```
1/1          0s 255ms/step
```

```
[10]: final_pred
```

```
[10]: array([[0.26608837],  
            [0.5626975 ],  
            [0.20315048]], dtype=float32)
```

```
[11]: import pandas as pd  
sub = pd.read_csv("/kaggle/input/equity-post-HCT-survival-predictions/  
↳sample_submission.csv")
```

```
[12]: sub['prediction'] = final_pred
```

```
[13]: sub
```

```
[13]:      ID  prediction  
0  28800    0.266088  
1  28801    0.562698  
2  28802    0.203150
```

```
[14]: sub.to_csv('submission.csv', index=False)  
print("Submission successfully.")
```

```
Submission successfully.
```

```
[ ]:
```